LESSON 23

来一个 push_back 放一个

vector 入门

学完以后，事先不知道有多少个数也能全存下来：来一个，往后面放一个。

读到 0 才停，要准备多少个盒子?

第 12 课的数组要先写好大小：int a[100]。题目先告诉我们 n，就知道要存几个。

可要是题目说“一直读，读到 0 才停”，事先不知道会有几个数。
准备 100 个，万一来了 200 个呢?

C++ 有一种会自己变长的数组，叫 **vector**。
来一个数，就往它后面放一个，它自己会长出新的位置。

push_back：往后面放一个

reverse.cpp

```
 6      vector<int> v;
 7      int x = 0;
 8      cin >> x;
 9      while (x != 0) {
10          v.push_back(x);
11          cin >> x;
12      }
13      int n = v.size();
```

后面和第 12 课一样，i 从 n - 1 倒着走，输出 v[i]。

开头要加 #include <vector>。最后的 0 只用来停，不放进去。

vector<int> v 准备一个放整数的 vector，一开始是空的。

v.size() 是现在有几个，v[i] 和数组一样，从 0 开始。

输入 3 1 4 0

```
4
1
3
```

输入 3 1 4 0：v 一点点变长

读到 3	push_back(3)	v 是 3，size 是 1
读到 1	push_back(1)	v 是 3 1，size 是 2
读到 4	push_back(4)	v 是 3 1 4，size 是 3
读到 0	0 != 0 不成立	不放，循环结束

新来的总是接在最后面：v[0] 是 3，v[1] 是 1，v[2] 是 4。

读完以后，v.size() 就是一共存了几个。

小测：输入 3 1 4 0，读完后 v.size() 是几？

1 4

2 3

3 100

4 2

答对了！3、1、4 放进去了，0 用来停，没放。

猜一猜：一上来就输入 0

reverse.cpp

```
13      int n = v.size();
14      for (int i = n - 1; i >= 0; i--) {
15          cout << v[i] << endl;
16      }
```



什么也没有，也不会出错。用 size() 当循环的边界，空的 vector 也能安全地走一遍。

vector 和数组，哪里一样，哪里不一样

`v[i]`	取数一样，下标从 `0` 开始
`v.size()`	自己知道有几个，不用另外记 `n`
`v.push_back(x)`	往后面放一个，数组做不到
`v[i]` 越界	和数组一样危险：下标要小于 `size()`

事先知道有几个，用数组或 `vector` 都行；不知道有几个，用 `vector` 更方便。

动手改一改

从 reverse 开始。读入不用动，改后面怎么用。

几个数，加起来多少?

一直读整数（-1000 到 1000，不是 0，最多 100 个），读到 0 停，0 不算。第一行输出读到几个数，第二行输出它们的和。

输入 3 1 4 0

```
3
8
```

输入 0

```
0
0
```

动手改一改

也从 reverse 开始。放进去之前，先过一道筛子。

只存偶数

读入同上。只把偶数存进 vector，读完以后按原来的顺序输出，每个占一行。

输入 3 8 1 4 0

```
8
4
```

输入 1 3 0

什么也没有

vector 也能 sort

sorted.cpp

```
14      sort(v.begin(), v.end());
```

输入 3 1 4 2 0

1 2 3 4

数组写 sort(a, a + n)，vector 写 sort(v.begin(), v.end())。

begin() 是开头，end() 是最后一个的后面，意思和 a、a + n 一样。

这也是一个固定写法，照着写。开头要同时加 #include <vector> 和 #include <algorithm>。

小测：v 里有 n 个数，最后一个是?

1 v[n]

2 v.size()

3 v[n - 1]

4 v[0]

答对了！和数组一样，n 个数的最后一个下标是 n - 1。

动手改一改

从 sorted 开始。排好以后，正中间的下标是几？

正中间的数

一直读整数，读到 0 停，0 不算，保证读到的是奇数个。排好以后，只输出正中间的那个数。

输入 3 1 4 2 5 0

3

输入 7 0

7

v[i] 越界，和数组一样危险

out.cpp（只阅读，不要运行）

```
6       vector<int> v;
7       v.push_back(3);
8       v.push_back(1);
9       cout << v[2] << endl;
```

v 里只有 2 个数，下标是 0 和 1，没有 v[2]。

vector 会长，但不是你一取它就长：本课用 push_back 让它变长，取 v[i] 不会多出一个。

取 v[2] 没有 error，结果说不准，和第 12 课的数组越界一样。下标要小于 size()。

读报错：size 后面少了括号

count.cpp（只阅读）

```
13      int n = v.size;
```

抓住后半句：**call it with no arguments**，不带参数调用它，也就是加一对空括号。

```
main.cpp:13:15: error: reference to non-static member function must be called; did you mean to call it with no arguments?
```

size() 是 vector 自带的函数，要加括号才会运行，交回有几个。

写成 v.size() 就好。push_back(x) 也一样，括号不能少。

找错：哪一行的顺序反了？

count.cpp（只阅读）

```
1       int x = 0;
2       cin >> x;
3       while (x != 0) {
4           cin >> x;
5           v.push_back(x);
6       }
```

点代码里的一行

这段想把读到的数都存进 v，输出个数和总和的部分这里没画。输入 3 1 4 0，应该是 3 和 8，屏幕上却是 3 和 5。点你认为顺序反了的那一行。

自己挑战

奇数站前面

输入 1 2 3 4 5 0，输出

1 3 5 2 4

输入 -3 -2 7 0，输出

-3 7 -2

一直读整数（-1000 到 1000，不是 0），读到 0 停。先按原来的顺序输出所有奇数，再按原来的顺序输出所有偶数，都在一行里，每个数后面空一格。

先自己写，卡住了再翻下一页。

卡住了？一次看一条提示

提示 1　准备两个 vector，一个放奇数，一个放偶数。

提示 2　读到一个数，用 % 判断它是奇数还是偶数，放进对应的那个。负数也能这样判断吗？想想 x % 2 != 0。

提示 3　读完以后，先把奇数的 vector 走一遍，再把偶数的走一遍。

记一本流水账

一直记数，记到 0 停，全都存进 vector，再从里面看出点什么。主题你自己定。

先做到

用 push_back 存下读到的数，输出一共记了几条。

选你喜欢的

每天步数、读书页数、零花钱、跳绳个数，都可以。

还想更厉害

再找出最多的一天，或者排好以后输出。

没有标准答案。写之前说清想要的结果，再试试一上来就输入 0。

和同桌换着玩

1

换着运行

运行同桌的程序，先记几条，再试试一上来就输入 `0`

2

说一个喜欢的

告诉同桌，你最喜欢这个程序哪里

3

提一个想法

问问同桌：数据都存下来了，还能看出什么？

听完再改一版。想一想、做出来、试一试、分享、再改，程序就是这样越做越好的。

说得清吗?

Q1 **什么时候用 vector 比数组方便?**

事先不知道有几个数，比如读到 0 才停的时候。

Q2 **push_back(x) 把 x 放在哪里?**

放在最后面，size() 多 1。

Q3 **vector 会长，能直接取 v[size()] 吗?**

不能，取 v[i] 不会让它变长，下标要小于 size()。

速查卡

```
#include <vector>  vector<int> v;
```

`v.push_back(x);`	往最后面放一个
`int n = v.size();`	现在有几个
`v[i]`	取数，下标 0 到 n - 1
`sort(v.begin(), v.end());`	从小到大排好
`while (x != 0)`	先放进去，再读下一个

这些后面再学

string 进阶

读一整行、拼接、截取一段

vector<string>

一串单词也能存进 vector

引用 &

把 vector 交给函数，又不复制一份

今天学会 vector：不用先定大小，来一个放一个。

谢谢！

喜欢这节课？欢迎来 GitHub 和我交流。

 cpp-for-youth

下一课：string 进阶